Table of Contents

Table of Contents 1
1. Introduction 2
1.1 What This Project Is 2
1.2 Why This Exists 2
1.3 Who Uses It and When 3
2. Problem Statement 4
2.1 The Core Problem 4
2.2 The UX Challenge 4
3. Literature Survey 5
3.1 Overview 5
3.2 AI and LLMs in Requirements Engineering 5
3.3 Full-Stack Architecture — The FARM Stack 5
3.4 Real-Time AI Streaming over WebSockets 6
3.5 Cognitive Load Theory and Enterprise UX 6
3.6 Summary Table 7
4. Proposed Plan of Work 10
4.1 What This Report Covers 10
4.2 How We're Approaching It 10
4.3 The User Journey, Step by Step 10
4.4 System Architecture 11
4.4.1 How the Layers Fit Together 11
4.4.2 Architecture Diagram 12
4.4.3 Design Decisions Worth Explaining 12
4.5 UX Design Strategy 13
4.5.1 The Design Principles Behind the Interface 13
4.5.2 The Key Screens and Modules 13
4.5.3 UX Risks and How We're Handling Them 14
5. AI Agents 15
5.1 The BRD Generation Agent 15
5.2 The TOC Generation Agent 15
5.3 Keeping Humans in the Loop 15
5.4 Risks and Mitigations 16
6. Project Schedule 17
6.1 Phase-by-Phase Plan 17
7. Conclusion 18
8. References 19

1. Introduction

1.1 What This Project Is

SDLC Studio came out of a straightforward frustration: writing Business Requirements Documents takes far too long, and the results are often inconsistent. The idea behind the platform is simple — give teams a single, guided workspace where AI handles the heavy lifting of structuring and drafting, while the people who actually know the project stay firmly in control of what gets written.

Under the hood, the platform is a modern web application built on four pillars. The frontend is React-based, chosen for its component model and ability to handle real-time content updates without a page reload. The backend runs on FastAPI — a lightweight Python framework that's fast enough to manage API requests, WebSocket connections, and AI agent orchestration without becoming a bottleneck. MongoDB stores everything: project metadata, individual BRD sections, version snapshots, and the document chunks fed to the AI. And sitting at the top is a set of Azure-hosted AI agents that do the actual content generation when triggered.

What ties it all together is the workflow. Rather than dumping users into a blank document and hoping for the best, SDLC Studio walks them through each stage — from uploading meeting notes and rough drafts, to generating a suggested document structure, to streaming a full BRD draft directly into an editable pane. It's designed to feel less like a documentation tool and more like a knowledgeable colleague who happens to type very quickly.

1.2 Why This Exists

Ask any business analyst or project manager about BRD creation and you'll hear a familiar story. It's slow, it's inconsistent, and it depends almost entirely on whoever is doing the writing that week. A document produced by one team looks nothing like one produced by another. Sections get skipped. Requirements get lost in email threads. And by the time the first proper draft exists, half the context from the original kickoff meeting has been forgotten.

These aren't edge cases — they're the norm. The BABOK Guide (IIBA, 2024) [12] reports that 81% of successful projects now follow industry-specific BRD formats, which tells you both how important standardisation has become and how rarely it's achieved in practice. The gap between what teams know they should do and what actually happens is where SDLC Studio is designed to operate.

Beyond the consistency problem, there's also a speed problem. Converting a set of meeting notes, a rough draft, and a handful of chat messages into a properly structured requirements document is genuinely time-consuming work — not because it's intellectually difficult, but because it's repetitive and formatting-heavy. That's exactly the kind of work AI is well-suited to assist with, provided the human still reviews and owns the output.

SDLC Studio addresses both issues at once: a guided interface that enforces good structure, and AI agents that turn raw inputs into a first draft fast enough to be useful.

1.3 Who Uses It and When

The platform is most valuable at the very beginning of a project, when requirements are still being shaped and there's a real risk that important context will get lost. That said, it's applicable across a range of teams and settings.

Enterprise product teams use it to make sure every new initiative starts with a properly structured requirements document, rather than a shared Google Doc that grows in unpredictable directions. Business analysts and PMO environments use it to bring consistency across projects that might otherwise be documented in completely different ways. Consulting firms find it useful for turning client discovery sessions into structured deliverables quickly — the kind of thing that used to require a full day of formatting work after a workshop.

Internal SDLC governance teams, where a formally signed-off BRD is a prerequisite for project approval, benefit from the version control and review workflow built into the platform. Every section can be reviewed, refined, and snapshotted before sign-off, which makes the governance process easier to manage and audit.

In short: if a team needs to produce a reliable, structured requirements document and wants to spend less time on the clerical parts of that process, SDLC Studio is designed with them in mind.

2. Problem Statement

The root of the problem is fragmentation. When a project kicks off, the context that matters — what was discussed in meetings, what the stakeholders actually want, what constraints exist — is scattered across meeting notes, email threads, Slack messages, and rough drafts. Nobody owns the job of pulling it all together, and by the time someone does, important things have been lost or misremembered.

The result is a BRD creation process that is slow, inconsistent, and expensive in terms of human time. Teams re-read the same notes multiple times, rewrite sections that were unclear the first time, and often discover mid-project that a key requirement was never captured properly in the first place.

2.1 The Core Problem

What's really needed is a single environment where all of that fragmented context can be collected, organised, and turned into a structured document — without requiring the person doing it to manually handle every formatting decision along the way. More specifically, the platform needs to:

- ☐ Bring all the reference material (meeting notes, drafts, chat exports) into one place in a way that the AI can actually use it.
- ☐ Propose a sensible document structure based on what's been provided, rather than starting from a generic template.
- ☐ Generate draft content for each section using the uploaded context, but keep the human in control of what stays and what gets changed.
- ☐ Track revisions properly, so teams can go back to any earlier state of the document without losing anything.
- ☐ Make the whole experience predictable and low-friction — something people will actually use on a real project, not just in a demo.

2.2 The UX Challenge

On the user experience side, the problem is just as real. Documentation tools that try to do too much at once are genuinely difficult to use. When you add AI capabilities on top of an already complex interface, the cognitive load on the user can become overwhelming — and people abandon tools that make them feel like they're doing more work, not less.

SDLC Studio has to handle several things that would normally compete for the user's attention: managing files, reviewing AI-generated structure, watching content stream into a live editor, and saving versions at the right moments. If the interface doesn't manage all of that carefully, the experience breaks down.

This is why Sweller's Cognitive Load Theory [6] is so central to the design philosophy here. The research is clear that when a system presents too many choices or too much information at once, people make worse decisions and are more likely to give up.

3. Literature Survey

3.1 Overview

This section surveys the research and industry literature most directly relevant to the SDLC Studio platform. It covers four interconnected areas: the use of large language models in requirements engineering, the FARM stack architecture the platform is built on, real-time AI streaming infrastructure, and cognitive load theory as it applies to enterprise UX design. Each area is grounded in recent academic and industry sources, and the connections to specific design decisions in SDLC Studio are made explicit throughout.

3.2 AI and LLMs in Requirements Engineering

The idea of using AI to help write requirements documents isn't new, but the capability has changed dramatically with the arrival of large language models. Zadenoori et al. (2024) [1] reviewed 74 primary studies published between 2023 and 2024 and found a clear shift in focus: researchers have moved away from using AI to classify existing requirements (a well-solved problem) and toward using it to generate, elicit, and validate new ones. Their key practical finding is that full AI autonomy is consistently rejected by practitioners — not because the output quality is poor, but because the people using these tools need to trust and own the output. This is the primary reason SDLC Studio is built around a review-before-apply model at every stage.

Vogelsang and Fischbach (2024) [2] go further, providing a guideline specifically for selecting and adapting LLMs for requirements tasks. Their recommendation is to use decoder-only, generative models (like GPT-4) for tasks that involve producing new text from a prompt — which is exactly what the TOC Agent and BRD Agent in SDLC Studio do. They also note the importance of structured prompting for reliable output, which informs the prompt assembly process in the BRD generation pipeline.

At a broader level, Hou et al. (2024) [3] analysed 395 papers on LLMs in software engineering across seven years and confirmed that AI-assisted documentation is now one of the most active and promising application areas. Their analysis validates the overall direction of the SDLC Studio project: there is strong evidence that LLMs can meaningfully reduce the time and effort involved in producing structured documents from unstructured inputs. A more recent study from ScienceDirect (2025) [14] further shows that GPT-4o specifically outperforms competing models on quality benchmarks for textual product requirements — supporting the decision to use Azure OpenAI as the backend AI provider.

On the commercial side, platforms like EltegraAI [9] report a 75% reduction in requirements gathering time for teams using AI-assisted BRD generation. This figure, while not independently verified, is consistent with the efficiency gains described in academic literature and provides a meaningful benchmark for evaluating SDLC Studio's value proposition once production data becomes available.

3.3 Full-Stack Architecture — The FARM Stack

SDLC Studio is built on the FARM stack — FastAPI, React, and MongoDB — a combination that has grown substantially in popularity since 2022 as teams look for lightweight, Python-friendly

alternatives to Node-based full-stack frameworks. Aleksendric (2022) [4] provides the most comprehensive treatment of this stack, covering the key architectural patterns for integrating all three layers including authentication, data modelling, and API design. This book is the primary reference for the SDLC Studio backend and database architecture.

The official MongoDB developer documentation [5] is the practical reference for the data persistence layer. MongoDB's document model is a natural fit for a platform that stores heterogeneous content — BRD sections have different structures from version snapshots, which differ from project metadata. The MongoDB Labs full-stack generator [16] also provides a production-ready reference for JWT-based authentication and Docker Compose configuration, both of which inform the SDLC Studio deployment design.

For teams looking to understand the full stack from first principles to production, the DataCamp FARM stack guide (2025) [13] is the most current resource, covering Celery for background task processing and Traefik for reverse proxying. These are patterns that will become relevant in SDLC Studio's post-UI/UX hardening phase, particularly for handling long-running generation tasks at scale.

3.4 Real-Time AI Streaming over WebSockets

The decision to stream BRD content progressively into the editor — rather than waiting for full generation to complete — is one of the most consequential UX decisions in SDLC Studio. It transforms what would otherwise be a frustrating waiting experience into something that feels active and responsive. The infrastructure to support this is non-trivial, and the literature is helpful in understanding the right approach.

Render Engineering (2024) [7] identifies three requirements for reliable real-time AI chat infrastructure: persistent WebSocket connections (not the stateless HTTP connections that serverless architectures rely on), uninterrupted LLM token streaming, and high-performance session management. Their conclusion — that serverless is fundamentally unsuited for this use case — directly informed the decision to run a persistent FastAPI server rather than a serverless function for the WebSocket proxy.

The practical implementation guide by Dpills (2024) [11] is the closest technical reference for the SDLC Studio streaming pipeline. It demonstrates the complete integration of OpenAI streaming, a FastAPI WebSocket endpoint, and a React frontend that renders the stream incrementally. The patterns described there — including client-side filtering of non-content messages and graceful handling of connection interruptions — are directly reflected in the SDLC Studio implementation.

3.5 Cognitive Load Theory and Enterprise UX

The UX design of SDLC Studio is grounded in cognitive load theory, and it's worth being explicit about why. When people are using a tool to do complex intellectual work — like structuring and reviewing a requirements document — the quality of their thinking is directly affected by how much mental effort the tool itself demands. A poorly designed interface doesn't just annoy users; it actively makes their work worse.

Sweller (1988) [6] established the foundational insight: human working memory is limited, and good design should reduce the cognitive demand placed on it. In practical terms, this means not

showing users information they don't need yet, not requiring them to remember things the system should remember for them, and making the consequences of actions clear and reversible. Every major UX decision in SDLC Studio can be traced back to one of these principles.

Think Design (2025) [10] translates these ideas into actionable interface patterns — chunking content into manageable pieces, using visual hierarchy to indicate importance, and providing consistent feedback at every step. Full Clarity (2024) [15] provides a useful typology of cognitive load types (intrinsic, extraneous, and germane) and how each should be managed in enterprise software. Their analysis of complex documentation tools is particularly relevant, as it confirms that the biggest source of cognitive load in these tools is typically not the task itself but the interface — menus that aren't where users expect them, actions with unclear consequences, and missing feedback during long operations.

Toptal's UX research [17] adds an enterprise-specific angle: for users who work with documentation tools every day, reducing cognitive overload isn't just about comfort — it's about preventing burnout and maintaining quality over time. A tool that feels effortful to use will either be abandoned or used badly. This is the standard against which SDLC Studio's interface is designed and measured.

3.6 Summary Table

Ref.	Author(s)	Title	Why It Matters for This Project	Year / Source
[1]	Zadenoori et al.	LLMs for Requirements Engineering: A Systematic Literature Review	Reviews 74 studies on LLM use in RE. Confirms that hybrid human-AI workflows outperform full automation — directly shaping the "AI suggests, human decides" model in SDLC Studio.	2024, arXiv:2509.11446
[2]	Vogelsang & Fischbach	Using LLMs for NLP Tasks in RE: A Systematic Guideline	Recommends decoder-only models (like GPT) for generation tasks. Guides the choice of Azure OpenAI for both the TOC and BRD agents.	2024, arXiv:2402.13823
[3]	Hou et al.	LLMs for Software Engineering: A Systematic Literature Review	Covers 395 papers on LLM use in SE. Validates the technical feasibility of AI-assisted documentation workflows like SDLC Studio.	2024, arXiv:2308.10620
[4]	Aleksendric, M.	Full Stack FastAPI, React, and MongoDB	Comprehensive guide to the FARM stack. Core architectural reference for how SDLC Studio's frontend, backend, and database layers are structured together.	2022, Packt Publishing

Ref.	Author(s)	Title	Why It Matters for This Project	Year / Source
[5]	MongoDB Developer Team	Introducing FARM Stack – FastAPI, React, and MongoDB	Official MongoDB guide with production-ready CRUD and auth patterns. Directly informs the backend API design and data persistence layer.	2023, MongoDB Docs
[6]	Sweller, J.	Cognitive Load Theory, Learning Difficulty, and Instructional Design	The foundational paper on how working memory handles complexity. Underpins the SDLC Studio UX strategy — particularly progressive disclosure and guided step-by-step workflows.	1988, Instructional Science
[7]	Render Engineering	Building Real-Time AI Chat: WebSockets, LLM Streaming, and Session Management	Identifies persistent WebSocket connections and LLM streaming as the two non-negotiables for a smooth real-time generation UX. Validates the SDLC Studio streaming architecture.	2024, Render.com
[8]	De Luca, G.	FastAPI Cookbook: WebSocket Communication & NoSQL Integrations	Advanced FastAPI patterns for WebSocket handling and MongoDB integration. Referenced in the design of the backend WebSocket proxy for BRD streaming.	2024, Packt
[9]	EltegraAI Platform	AI BRD Generator — Platform Overview	A live commercial platform claiming 75% reduction in requirements gathering time via AI. Used as an industry benchmark for evaluating SDLC Studio's value proposition.	2024, eltegra.ai
[10]	Think Design	Cognitive Load in UX Design: Key Strategies for Management	Practical application of Sweller's theory to UI design — chunking, visual hierarchy, and reducing noise. Applied directly to the SDLC Studio workspace layout.	2025, Think Design Blog
[11]	Dpills (Dev.to)	Create a Streaming AI Assistant with ChatGPT, FastAPI, WebSockets and React	Step-by-step implementation guide for streaming LLM output over WebSockets to a React UI. Close technical reference for the BRD generation pipeline.	2024, DEV Community

Ref.	Author(s)	Title	Why It Matters for This Project	Year / Source
[12]	IIBA	Business Analysis Body of Knowledge (BABOK Guide)	Industry standard for business analysis. 2024 data shows 81% of successful projects use industry-specific BRD formats — justifying SDLC Studio's structured, template-first approach.	2024, IIBA
[13]	DataCamp	FARM Stack Guide: Building Full-Stack Apps with FastAPI, React & MongoDB	Production-grade FARM stack guide covering auth, task queues (Celery), and CI/CD. Relevant to post-UI/UX hardening phases of SDLC Studio.	2025, DataCamp
[14]	ScienceDirect	Advancing Requirements Engineering with Large Language Models	GPT-4o outperforms other models on textual requirements quality benchmarks. Supports the decision to use Azure OpenAI as the backend AI provider.	2025, ScienceDirect
[15]	Full Clarity	Reducing Cognitive Overload in UX Design	Practical strategies for managing intrinsic and extraneous cognitive load in complex enterprise tools. Referenced in the UX risk analysis and workspace design decisions.	2024, fullclarity.co.uk

4. Proposed Plan of Work

4.1 What This Report Covers

This report focuses specifically on getting the UI/UX baseline right. That means everything from the first wireframe to a working, integrated interface that a real user could walk through end-to-end. The work covered here includes:

- Translating the functional and non-functional requirements into actual frontend workflows.
- Designing the information architecture — how the app is organised, what pages exist, and how users move between them.
- Building the component-level interaction model, including all the states a user might encounter (loading, empty, error, success).
- Connecting the frontend to the backend APIs that drive AI behaviour — specifically the TOC generation endpoint and the WebSocket stream for BRD content.
- Designing the hooks where AI assistance becomes visible in the UX: the generate button, the streaming editor, the TOC modal.

What's explicitly not in scope for this report: production hardening, security audits, performance optimisation at scale, DevOps and deployment, and any features planned for phases after the UI/UX baseline.

4.2 How We're Approaching It

The team is working through an iterative, UI-first process — meaning the user experience shapes the development, not the other way around. The six stages of the approach are:

- ☐ Define — Map out who the users are, what they're trying to do at each stage, and what success looks like. This produces the user flow diagrams and use case documentation.
- ☐ Architect — Translate the user flows into a concrete screen-level architecture: what components exist, how they're composed, and what shared state they depend on.
- ☐ Implement — Build the React component library and set up the global AppContext that manages authentication state, project data, and the content stream.
- ☐ Integrate — Connect the frontend to live backend APIs. This is where the interface stops using mock data and starts behaving like a real application.
- ☐ Enable AI — Add the specific interaction points where the AI agents become visible: the TOC generation trigger, the stream rendering in the editor, and the state changes that indicate generation is in progress.
- ☐ Refine — Go back through every screen and interaction with fresh eyes, focusing on feedback clarity. Does the user always know what's happening? Can they recover from mistakes easily?

4.3 The User Journey, Step by Step

Here's how a typical session in SDLC Studio plays out from a user's perspective:

- ☐ Login or Register — The user arrives at the auth page and either logs into an existing account or creates a new one. Straightforward, with validation that catches errors before they become frustrating.
- ☐ Dashboard — After logging in, the user sees their projects. They can open an existing one or start something new. Project cards show just enough metadata to orient the user without overloading them.
- ☐ Workspace — Selecting a project opens the core working environment. This is where most of the session is spent.
- ☐ Context Upload — Before generating anything, the user uploads the relevant reference files. These get categorised automatically (Meeting Notes, Drafts, Transcripts, Chat Logs) and become the source material for the AI.
- ☐ TOC Generation — The user triggers the structure generation. The AI suggests a Table of Contents based on the project context. The user reviews it, reorders sections, edits descriptions, and adds any specific instructions before confirming.
- ☐ BRD Generation — With the structure confirmed, the user launches content generation. The AI begins writing each section, streaming the output live into the document editor.
- ☐ Review and Edit — Once generation completes, the full BRD draft is in the editor. The user reads through it, makes changes directly, and saves version snapshots as they go.
- ☐ Download — When the document is ready, the user can export it and share it with stakeholders.

4.4 System Architecture

4.4.1 How the Layers Fit Together

The platform is built as a set of distinct services that each have a clear, well-defined job. This separation makes the system easier to maintain and easier to scale independently if one part needs to grow faster than another.

The React frontend is responsible for everything the user sees and interacts with. It handles authentication state, renders the project dashboard, runs the Workspace editor, and manages the modal-based workflows (TOC editing, version history, file uploads). It communicates with the backend over standard REST for most operations, and over a WebSocket connection specifically for the AI content stream.

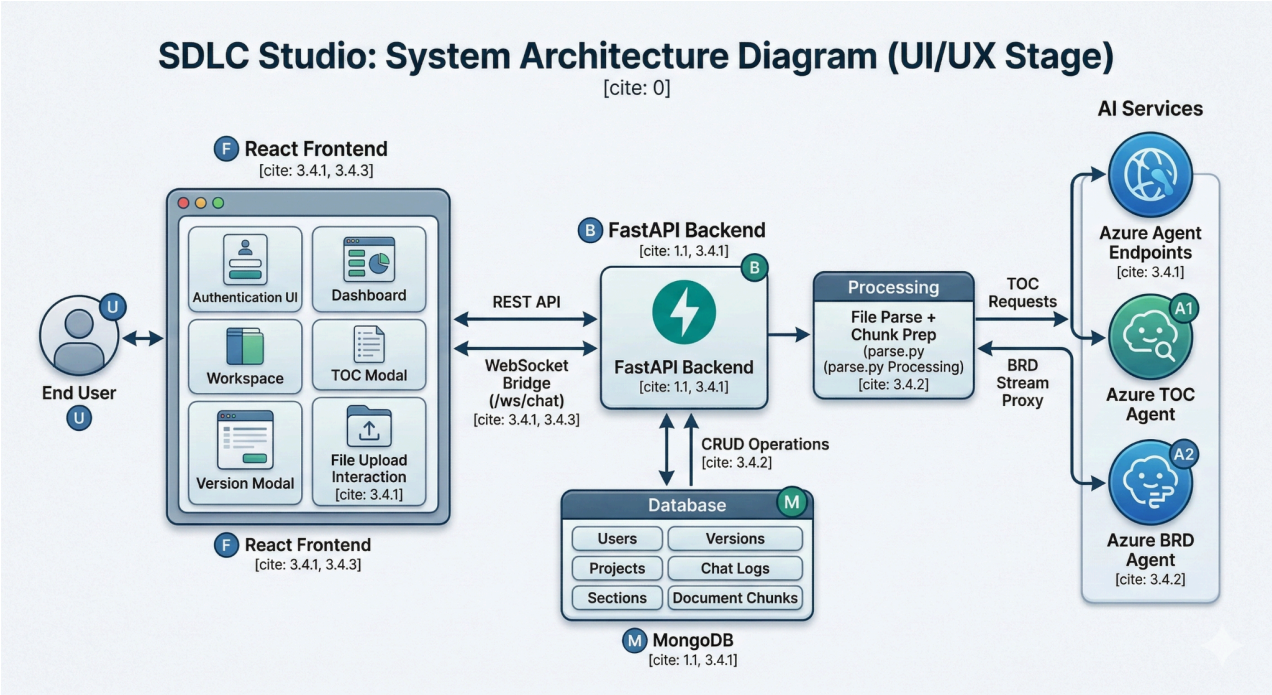
The FastAPI backend is the application's central nervous system. It receives API requests from the frontend, handles all business logic, manages the WebSocket bridge that proxies the AI agent stream, and coordinates file ingestion and parsing. It also owns the section and version persistence layer.

MongoDB stores the data. Its document model is a natural fit for a system that deals with structured-but-variable content: BRD sections look different from project metadata, which looks different from version snapshots. MongoDB handles all of this without requiring a rigid schema.

The Azure AI agents sit outside the core application as separate, containerised services. The TOC Agent takes a project context and returns a structured JSON document — a suggested

section hierarchy. The BRD Agent takes the confirmed structure plus all uploaded context and streams the actual document content back to the frontend via the WebSocket proxy.

4.4.2 Architecture Diagram



4.4.3 Design Decisions Worth Explaining

A few architectural choices have a direct impact on the user experience, and it's worth explaining why they were made:

- **Protected routing:** Every route beyond the auth page checks for a valid session before rendering. Users who aren't logged in get redirected cleanly rather than hitting a broken page.
- **Centralised state via AppContext:** Rather than threading props through multiple layers of components, a single context object tracks the logged-in user, the current project, uploaded files, and the live stream text. This makes it much easier to keep the UI consistent as state changes.
- **WebSocket isolation for AI streaming:** The high-frequency message stream from the AI agent is handled by a dedicated layer that never blocks the main UI thread. The editor stays interactive and responsive even when new content is arriving rapidly — a pattern validated by industry research on real-time AI infrastructure [7].
- **Modal-driven flows for complexity:** Any workflow that would clutter the main workspace (creating a project, editing the TOC, managing versions) lives in a modal. The user's focus stays on the document, not on the tooling around it.
- **Atomic backend updates:** The backend doesn't do partial saves. When a section is updated or a version is snapshotted, it's a complete, consistent write. This is what makes the version history trustworthy.

4.5 UX Design Strategy

4.5.1 The Design Principles Behind the Interface

The interface design isn't arbitrary — it's grounded in research on how people actually process information and make decisions when using complex tools. The five principles that run through every screen are:

- **Guided workflow:** The platform leads users from one stage to the next with clear calls to action. There's no moment where the user is faced with a blank screen and no indication of what to do. This is a direct application of cognitive load reduction — fewer choices at each step means less mental effort and fewer mistakes.
- **Visible system status:** If the application is doing something that takes time, the user always knows. Spinners appear during API calls. Toast notifications confirm when something has been saved. Buttons change state — or disable entirely — while a task is running. This builds trust. Users don't worry that their action was ignored.
- **Progressive disclosure:** The workspace doesn't show everything at once. Version history, advanced file settings, and generation options are tucked into sidebars and modals that only appear when the user actively needs them. The main editing area stays clean and focused.
- **Recoverability:** Users can make mistakes without consequence. Content is saved in the background automatically. Explicit version snapshots let users roll back to any earlier state. The system is designed to feel safe — to encourage experimentation rather than make users anxious about every edit.
- **Consistency:** The same patterns appear in the same places throughout the application. Project cards on the dashboard, action buttons in the workspace, modal headers — they all look and behave the same way. This means users only have to learn the interface once.

4.5.2 The Key Screens and Modules

The frontend is structured around a small number of high-impact modules, each of which owns a specific part of the user journey:

- **Auth Screen:** Minimal by design. A single form that toggles between login and registration. Client-side validation catches errors immediately, and loading states prevent accidental double-submissions.
- **Dashboard:** The launchpad. Users see their projects as cards, with enough information (title, last modified, status) to orient them quickly. The main action is opening a project or creating a new one.
- **Workspace:** The core environment. It's split into three areas — the Document Editor Pane (where the BRD lives), the Left Sidebar (for file uploads and the TOC launcher), and the Action Bar (for switching between read and edit modes, saving versions, and downloading). Each area has a single clear purpose.
- **TOC Modal:** The structure editor. Once the AI has suggested a table of contents, this modal lets the user reshape it: drag sections to reorder them, edit section titles and

descriptions, add specific instructions for how each section should be written. Nothing gets applied to the document until the user confirms.

- **Prompt Cards:** Visual representations of uploaded files, grouped by category. They make it obvious what context is available to the AI, and they're the mechanism by which users control what gets fed into the generation prompt.

4.5.3 UX Risks and How We're Handling Them

UX Risk	How We Handle It
Long-running AI generation tasks	A persistent loading indicator stays visible in the editor while generation is running. Modals close automatically so users can continue working in the background without being blocked.
Risk of losing unsaved work	Users can manually save or snapshot a version at any point. On top of that, the system quietly saves content in the background at regular intervals, so an accidental browser close or dropped connection won't wipe out hours of work.
AI input becoming too complicated	Files are uploaded through clearly labeled Prompt Cards (e.g., Meeting Notes, Drafts), and each section in the TOC modal has its own focused instruction field — so users always know exactly what they're feeding the AI.

5. AI Agents

5.1 The BRD Generation Agent

The BRD Agent is the part of the system that actually writes. It's connected to the frontend through a WebSocket proxy endpoint on the backend (/ws/chat), which means content flows into the editor continuously as it's generated — not in one big block at the end. This matters a lot for the user experience: watching a document take shape in real time feels fundamentally different from waiting for a spinner to finish and then seeing a wall of text appear.

When a user triggers generation, the backend assembles a carefully structured prompt that includes the confirmed document structure and all the uploaded reference material. That prompt goes to the Azure BRD Agent, which begins streaming its response back. The frontend filters the stream to strip out any metadata or status messages, so only clean, readable content reaches the editor.

From a UX perspective, the impact is immediate: the Generate button transitions visibly to "Generating...", a progress indicator appears in the editor, and the first content starts arriving within seconds. When generation finishes, everything in the editor is fully editable — the AI's draft is a starting point, not a final product.

Research on LLM use in requirements engineering [1] consistently shows that practitioners want AI to accelerate drafting, not replace human judgement. That's exactly the balance this agent is designed to strike.

5.2 The TOC Generation Agent

Before a single word of BRD content gets written, the user needs a document structure. That's the job of the TOC Agent. It's triggered through a REST endpoint (/api/generate-toc) rather than a WebSocket, because the output — a structured JSON object representing the section hierarchy — needs to arrive as a complete, valid payload before the modal can render it.

The agent takes the project title and any initial context the user has provided, and returns a suggested table of contents: section titles, brief descriptions of what each section should contain, and initial generation instructions that the user can edit. This pre-populated structure is what appears in the TOC modal when the user clicks "Generate Structure."

The key design decision here is that nothing happens automatically. The AI's suggestion appears in the modal, the user reviews it, makes whatever changes feel right, and only then applies it to the document. Vogelsang and Fischbach (2024) [2] make the point that LLMs are most effective in RE workflows when their output is treated as a high-quality first draft rather than a finished artefact. That's exactly how the TOC Agent is positioned.

5.3 Keeping Humans in the Loop

The guiding principle for both agents is simple: the AI is here to make the human faster, not to replace the human's judgement. In practice, this means a few concrete things.

Nothing generated by an AI agent is applied to the document without the user's explicit approval. The TOC suggestion sits in a modal until the user confirms it. The BRD draft streams into an

editable pane — it's always modifiable, never locked. Every major generation action triggers an automatic version snapshot before it runs, so there's always a clean restore point if the output isn't what the user wanted.

This hybrid model — AI for speed, human for quality control — is the approach that most current research recommends for AI-assisted software engineering [3]. It's also, frankly, what users actually trust. A system that makes decisions on their behalf without asking is a system they'll stop using after the first mistake.

5.4 Risks and Mitigations

Risk	How It's Handled
TOC agent returns garbled or non-JSON output	The backend validates and extracts JSON before anything reaches the UI. If the response is malformed, it falls back gracefully rather than crashing the modal.
WebSocket stream sends duplicate or noisy messages	The frontend filters out non-content payloads (like status pings or echo frames) before rendering, so only clean, readable text appears in the editor.
External AI service becomes unavailable mid-generation	A clear, friendly error message is shown to the user. The WebSocket connection is built to support retry attempts, so users can pick up where they left off without restarting from scratch.

6. Project Schedule

6.1 Phase-by-Phase Plan

Phase	When	What We're Doing	What We Deliver
Phase 1 — Discovery & Requirements	Week 1	Map out the BRD workflow, identify who does what, and set boundaries for the UX scope.	Requirements notes, User Flow Draft
Phase 2 — Information Architecture	Week 2	Lock in the app's page structure, navigation routes, and how components are grouped.	IA Map, Screen Blueprint
Phase 3 — Core UI Build	Weeks 3–4	Build the Auth screens, Dashboard, and the Workspace skeleton with real interactions.	Functional Frontend Skeleton
Phase 4 — Guided UX & Modals	Week 5	Wire up the TOC modal, file upload flow, notification toasts, and loading state indicators.	Guided Interaction Layer
Phase 5 — AI Integration	Week 6	Connect the TOC suggestion agent, hook up BRD generation launch, and render the streamed output live.	AI-Assisted UX Flow
Phase 6 — Stabilisation & Sign-off	Week 7	Walk through the full journey end-to-end, catch any rough edges, and get sign-off on the UI/UX baseline.	UI/UX-Ready Baseline

The seven-week timeline is deliberately structured so that each phase builds directly on the previous one. We don't start building AI integrations until the core UI is stable, and we don't start the stabilisation phase until all the integrations are in place. This sequencing reduces the risk of rework and makes it easier to identify which part of the system is responsible when something doesn't work as expected.

7. Conclusion

The SDLC Studio platform, as it stands at the end of the UI/UX baseline stage, does what it set out to do. It takes a process that is typically slow, inconsistent, and mentally demanding — writing a Business Requirements Document from a collection of raw project inputs — and makes it faster and more structured without taking control away from the people who know the project best.

The interface is designed around how people actually work, not how a tool designer imagines they work. The guided workflow, the progressive disclosure, the real-time feedback, the recoverability through version history — these aren't cosmetic choices. They're the direct application of decades of research on cognitive load and interface design [6, 10, 15], translated into practical decisions about where to put a button or when to show a spinner.

The AI agents are integrated as tools, not authorities. The TOC Agent and BRD Agent accelerate the most time-consuming parts of the documentation process. But every suggestion they make is modifiable, every piece of content they generate is editable, and every major action they take is reversible. This is the model that current research consistently recommends for LLM-assisted requirements engineering [1, 2, 3], and it's the model that practitioners actually trust.

What comes next — production hardening, performance scaling, security audits, and the feature roadmap beyond the UI/UX baseline — is outside the scope of this report. But the foundation is solid. The architecture is clean, the interaction design is coherent, and the platform is genuinely usable by the people it's built for. That's the right starting point.

8. References

[1] Zadenoori, M. A., et al. (2024). Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv preprint arXiv:2509.11446. https://arxiv.org/abs/2509.11446

[2] Vogelsang, A., & Fischbach, J. (2024). Using Large Language Models for Natural Language Processing Tasks in Requirements Engineering: A Systematic Guideline. arXiv preprint arXiv:2402.13823. https://arxiv.org/abs/2402.13823

[3] Hou, X., et al. (2024). Large Language Models for Software Engineering: A Systematic Literature Review. arXiv preprint arXiv:2308.10620. https://arxiv.org/abs/2308.10620

[4] Aleksendric, M. (2022). Full Stack FastAPI, React, and MongoDB: Build Python web applications with the FARM stack. Packt Publishing. ISBN: 9781803231822. https://www.packtpub.com/en-us/product/full-stack-fastapi-react-and-mongodb-9781803231822

[5] MongoDB Developer Team. (2023). Introducing the FARM Stack – FastAPI, React, and MongoDB. MongoDB Developer Documentation. https://www.mongodb.com/developer/languages/python/farm-stack-fastapi-react-mongodb/

[6] Sweller, J. (1988). Cognitive Load Theory, Learning Difficulty, and Instructional Design. Instructional Science, 12(4), pp. 257–278. https://lawsofux.com/cognitive-load/

[7] Render Engineering. (2024). Building Real-Time AI Chat: Infrastructure for WebSockets, LLM Streaming, and Session Management. https://render.com/articles/real-time-ai-chat-websockets-infrastructure

[8] De Luca, G. (2024). FastAPI Cookbook: Custom Middleware, WebSocket Communication, SQL and NoSQL Integrations. Packt Publishing. https://bookauthority.org/books/new-fastapi-books

[9] EltegraAI. (2024). AI BRD Generator – Platform Overview. https://www.eltegra.ai/brd-generation

[10] Think Design. (2025). Cognitive Load in UX Design: Key Strategies for Management. https://think.design/blog/cognitive-load-in-ux-design/

[11] Dpills. (2024). Create a Streaming AI Assistant with ChatGPT, FastAPI, WebSockets and React. DEV Community. https://dev.to/dpills/create-a-streaming-ai-assistant-with-chatgpt-fastapi-websockets-and-react-3ehf

[12] International Institute of Business Analysis (IIBA). (2024). Business Analysis Body of Knowledge (BABOK Guide). https://www.iiba.org/babok-guide/

[13] DataCamp. (2025). FARM Stack Guide: How to Build Full-Stack Apps with FastAPI, React & MongoDB. https://www.datacamp.com/tutorial/farm-stack-guide

[14] ScienceDirect. (2025). Advancing Requirements Engineering with Large Language Models. ISSN: 2212-8271. https://www.sciencedirect.com/science/article/pii/S221282712500873X

[15] Full Clarity. (2024). Reducing Cognitive Overload in UX Design. https://fullclarity.co.uk/insights/cognitive-overload-in-ux-design/

[16] MongoDB Labs. (2023). Full Stack FastAPI and MongoDB – Base Project Generator. GitHub Repository. https://github.com/mongodb-labs/full-stack-fastapi-mongodb

[17] Toptal Designers. (2022). 4 UX Tips to Reduce Users' Cognitive Overload and Burnout. Toptal Design Blog. https://www.toptal.com/designers/ux/cognitive-overload-burnout-ux